

The integrated system

No new material — seven weeks of parts, running at the same time, on purpose

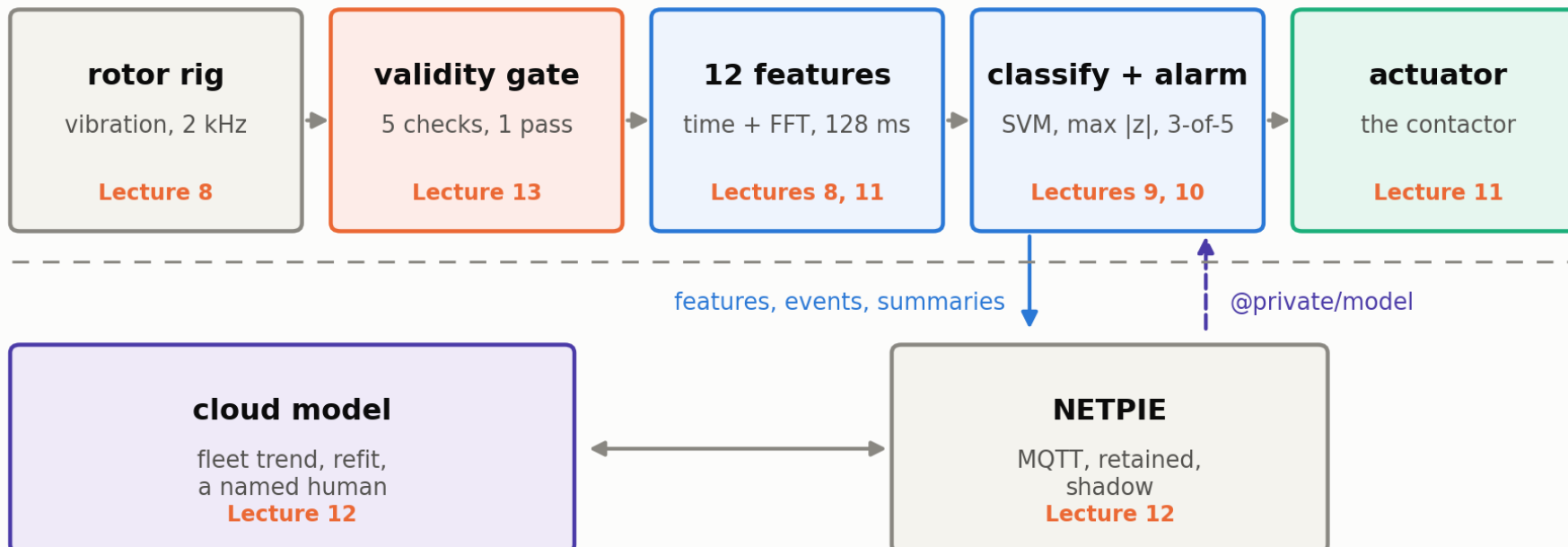
- Seven weeks, one system — and the one line that divides it
- Today's run: 22 minutes, a heavy cut, a broken sensor, a spill, three model updates, no WiFi
- What you must observe — 12 checkpoints, and why four of them are refusals
- The five ways this system can be wrong, and which lecture defends each
- Every design decision we made, and what it cost
- Where the safety mechanism you trust most stops working
- What a real plant needs that this course did not give you

Notebook: `iaiiot26_Lect14_nb.ipynb`

Seven weeks, one system

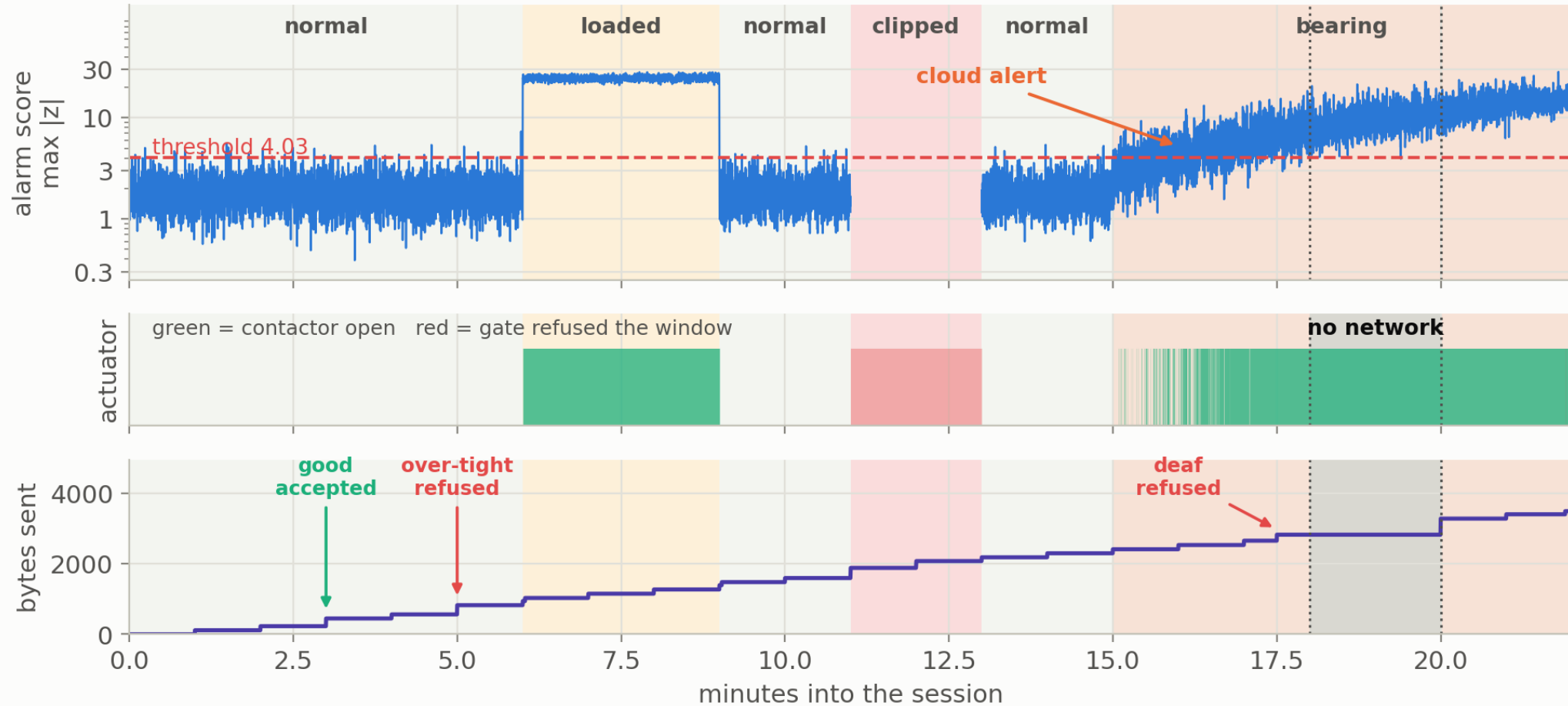
- Every box below was built and tested in one of the last six lectures; nothing today is new except the wiring.
- The dashed line is the whole architectural argument: what is above it must keep working when everything below it is gone.

Everything above the dashed line must work with the WiFi unplugged. Everything below it is how the machine gets better.



If you can only remember one picture from this course, remember the line — not the boxes.

Today's run, minute by minute



Two of the four disturbances are not faults. A system that alarms on all four is not a monitoring system.

What you have to observe

- 12 claims about the system, each of which this run either supports or refutes. Four of them are about the system declining to do something.

what you must observe	what this run did	obs.
quiet start	0.0%	PASS ☐
good update accepted	accepted: committed	PASS ☐
the flooding spec refused	refused: shadow test: would raise on 100% of	PASS ☐
the deaf spec refused	refused: shadow test: deaf: raises on only 0	PASS ☐
heavy cut raises the alarm	raised 100%, driver rms	PASS ☐
sensor fault caught by the gate	100% rejected	PASS ☐
actuator held during the sensor fault	0 transitions	PASS ☐
the spall raises the alarm	raised 100%, driver e_hi	PASS ☐
the cloud alerts, and only on the spall	alert at 16.0 min	PASS ☐
the device keeps deciding with no network	1830 windows decided	PASS ☐
nothing published while the link is down	3 queued	PASS ☐
queued reports arrive on reconnect	0 still queued at the end	PASS ☐

This run: 12 of 12. The empty column is yours to fill in this afternoon.

The five ways this can be wrong

- Every defence in this system was built for one of these rows, and each one is useless against the others.

what can go wrong

the machine is breaking
a bearing spall, an imbalance

the sensor is broken
stuck, clipped, wrong gain

the model is wrong
fitted while loaded, or deaf

the link is gone
WiFi down, broker down

nothing is wrong
a heavier cut, a cold start

what stops it, and where it lives

features + alarm, on the device
Lectures 8-10

the validity gate, before features
Lecture 13

the shadow test, before commit
Lecture 13

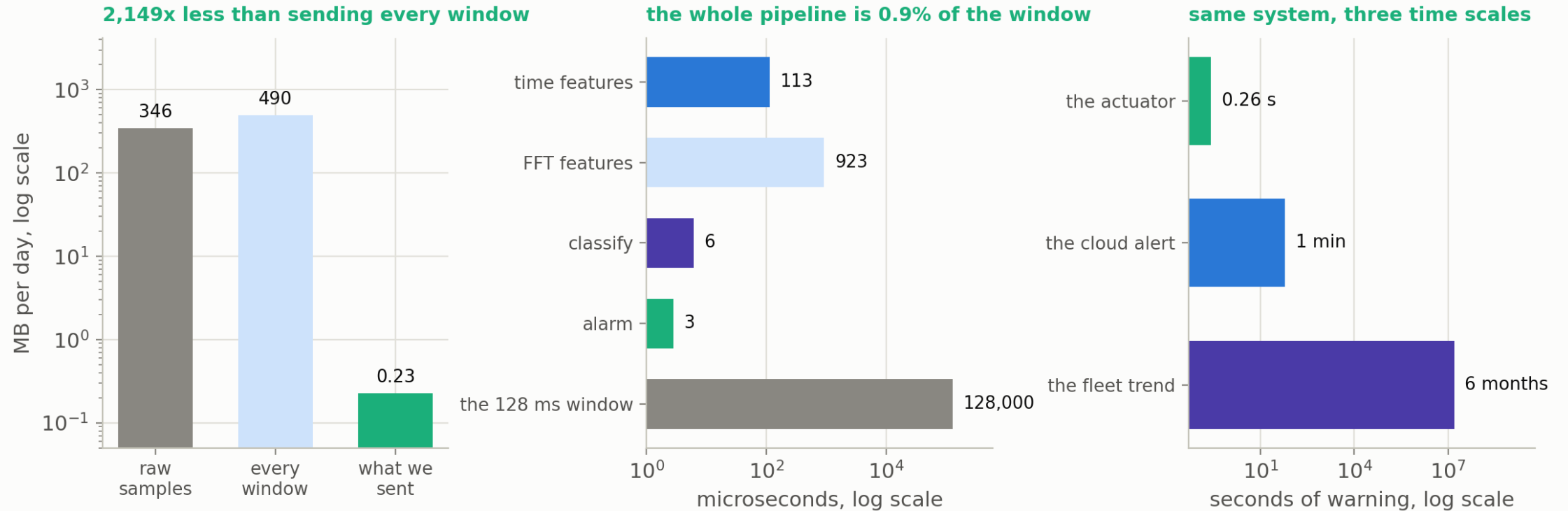
decide locally, queue the reports
Lectures 11-12

persistence, dwell, the driver name
Lecture 10

The last row is the one that gets systems switched off: a system that cries wolf is removed by the people it protects.

The numbers, seven weeks on

- Three questions students asked in week eight, answered by measurement rather than by opinion.



The device spends 0.9 % of its window computing, and sends 0.23 MB a day. Neither was the hard part.

Every decision, and what it cost

the decision	where	what it bought	what it cost
12 features, not the raw wave	Lecture 8	512 -> 48 bytes per window	a feature nobody thought of is gone
a linear SVM, not a forest	Lecture 9	39 numbers, 6 us per window	curved boundaries go unmodelled
max z , not a class score	Lecture 10	detects faults never trained on	it cannot say what is wrong
3-of-5 persistence	Lecture 10	nuisance machines 25% -> 1.5%	0.022 of recall, 256 ms slower
FFT on the device	Lecture 11	+0.011 accuracy over time-only	8.5x the cost, still inside budget
decide at the edge	Lecture 11	works with the WiFi unplugged	the cloud sees less
on-change + counting heartbeat	Lecture 12	0.23 MB/day, 1 min latency	no waveform to re-examine later
validate, stage, verify, commit	Lecture 13	2 of 2 bad specs refused today	one deaf to a NEW fault gets in
a named human approves a refit	Lecture 13	somebody owns the consequence	slower than an automatic pipeline

You should be able to defend any row of this table — and name the plant where the other choice would win.

Where the shadow test stops working

- The deaf specification was refused because the device had kept windows from a fault it was already detecting — and failed to raise on them.

```
pushed at 15.0 min  ACCEPTED  <- the kept "alarmed" windows are still
pushed at 16.0 min  ACCEPTED      the HEAVY CUT: amplitude, not impulse
pushed at 16.5 min  refused    <- the buffer has turned over to spall
pushed at 17.5 min  refused      "raises on only 0% of the windows
                                that alarmed last time"
```

What it does defend against

A specification that is deaf to a fault this device has ALREADY seen and raised on. That is a strong test, and no arithmetic check can do it.

What it cannot defend against

A specification deaf to a fault this device has NEVER seen — which, on a new machine, a new fault type, or the first unit of a fleet, is every fault.

Testing against your own history cannot cover what your history does not contain. The fix is organisational, not arithmetic.

What would change in a real plant

- You were told this question in Lecture 13 so you could think about it. These are the answers that come up every year, and each is right.

The sensor is the project

Mounting, cabling, grounding and the mounted resonance decide what you can detect, and none of it is software. A stud-mounted accelerometer and a magnet-mounted one are different instruments.

Labels come from the maintenance system

Not from a simulation. Somebody has to connect a work order to a date and a machine, and that link is usually missing, wrong, or three weeks late.

The alarm has to go somewhere

A contactor, a CMMS ticket, a person on a shift rota. An alarm with no owner is removed within a month, and it is removed by the people it was protecting.

Security is not optional

Certificates, device identity, signed updates and secure boot. Our checksum catches corruption; it does not catch anybody who wants in.

Every one of these is somebody's whole job. None of them is machine learning.

What we did not teach you

we taught you the shape of

- one machine, one sensor, one fault family
- a linear model you can read as numbers
- a threshold you derived from your own data
- an update path that can refuse an update
- a device that decides with no network

a plant will also need

- real vibration, with real mountings and cabling
- many machines, and the fleet statistics from them
- deep models, quantisation, a C or TFLite runtime
- certificates, identity, signed updates, secure boot
- MTBF, spares, the maintenance planner's diary
- and somebody on call when it raises at 3 a.m.

You have the shape of the problem. You do not yet have a plant, and the difference is mostly not code.

The exam, and this afternoon

The exam covers Lectures 8–14

The first half of the course is examined separately. The published bank has 46 questions with model answers; the paper draws from it. Expect to defend a design decision, read a confusion matrix, and say what you would measure next.

What will NOT be asked

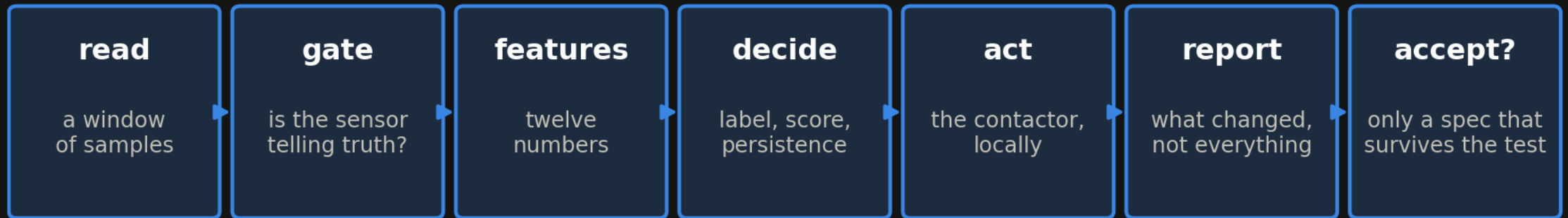
Memorised formulas, library API details, or the exact constants in my rig. If a number matters, it will be given to you. What is asked is why it matters.

Lab 14 — each group runs its own chain

Your Wokwi device, your NETPIE account, your model from Lab 12 and your gate from Lab 13. I will announce the script minute by minute; you run it and fill in the observation sheet as it happens. Two things are not announced in advance: when the sensor fault starts, and when the network goes. Hand the sheet in at the end of the session — it is the deliverable.

A checkpoint with no observation recorded scores zero, even if your system did the right thing.

The loop, and the six rules that keep it honest



every box runs on the ESP32; only the last two need a network

1. Decide on the device. Reporting may need a network; acting may not.
2. Check the sensor before you trust the model — a model cannot tell a quiet machine from a dead cable.
3. Hold the actuator when the input is bad, and say so out loud.
4. Report the driver name, not just the score. The number is 'over threshold'; the name is actionable.
5. Never apply a specification you have not run — against a quiet stretch and a stretch that alarmed.
6. Write down what the system cannot do. The limits you do not test for are still there.

Generated by Claude and customized by



dew.ninja 2026